

Process Documentation — ETL & ELT (Data Flow)

ID: Dokumen ini menelusuri **alur data tahap-demi-tahap**: untuk tiap langkah dijelaskan *input* → *kode yang mengerjakan* → *output* beserta **angka hasil run nyata**. Referensi komponen per-file ada di [DOKUMENTASI LENGKAP .pdf](#).

EN: This document traces the **data flow stage by stage**: for each step we show *input* → *the code that does it* → *output* with **real run numbers**. Per-file reference is in [DOKUMENTASI LENGKAP .pdf](#).

BAGIAN A — JALUR ETL / PART A — ETL PATH

Input: UCI URL (zip→.xls) + FRED API (4 seri, 2005).
Code / Code: etl_pipeline/extract.py (extract_etl_source1/2).
Proses / Process (ID): unduh apa adanya, **tanpa cleaning**; catat metadata per sumber.
Process (EN): download as-is, **no cleaning**; log per-source metadata.
Output (nyata/real): raw/uci_credit_card.xls (30,000×25); raw/fred_<id>_2005.json ×4 (12 observasi/seri). Log: etl_pipeline/logs/extract.log.

Input: `raw/uci_credit_card.xls` (30k).
Code / Code: `synthetic/train_ctgan.py`.
Proses (ID): latih CTGAN, generate 150k nasabah dengan *conditional sampling* untuk menjaga rasio default ~22%, seed 42. Output **disalin ke** `raw/` sebagai **source 1** lalu CTGAN **di-skip** pada run berikutnya (hemat memori, hindari OOM Hive).
Process (EN): train CTGAN, generate 150k clients via conditional sampling (preserve ~22% default), seed 42; copy output to `raw/` as **source 1**, then CTGAN is **skipped** on later runs.
Output (real): `raw/synthetic_credit_clients.csv` — 150,000 baris, balance {0:0.7788, 1:0.2212}.

1. **Load + standardisasi nama** → `snake_case`.

```
python df = spark.read.csv(str(raw_synth), header=True, inferSchema=True) # source 1 df =  
standardise_columns(df) # "default payment next month" -> default_payment_next_month
```

2. **Cleaning:** dedup `id`; isi missing numerik=0; **IQR cap** kredit; **Z-score flag** makro; clamp `age` ∈ [18,100].
3. **Normalisasi makro (min-max) + fitur klien:** `avg_delay_months`, `num_months_delayed`, `repayment_gap`, `total_bill/total_pay` (dibuat di `build_dim_client`).
4. **Unpivot** → **grain bulanan** (Apr–Sep 2005) + fitur bulanan `credit_utilization`, `payment_ratio`; **join makro** via `date_key`.
`python for dk, month, pay_col, bill_col, pay_amt_col in MONTH_MAP: # 6 bagian → union`
`parts.append(df.select(F.lit(dk).alias("date_key"), F.col(pay_col).alias("pay_status"), ...))` `fact = fact.join(F.broadcast(macro), on="date_key", how="left")`
5. **Validasi 6 aturan** → tulis `warehouse/staging/validation_report.json`.

Output (real): `warehouse/staging/` → `fact_credit_monthly` **900,000** baris (150k×6),
`dim_client` 150,000, `dim_date` 12, `dim_macro` 12. Validasi **6/6 PASS** (`default_rate`=0.2212).

A4. Load (Hive star schema) + 8 query

Input: `warehouse/staging/*.parquet`.

Kode / Code: `etl_pipeline/load.py` + `warehouse/etl_analytical_queries.sql`.

Proses (ID): `DROP DATABASE ... CASCADE` → `CREATE` (idempoten) → `saveAsTable` tiap tabel →
`ALTER TABLE ... ADD CONSTRAINT PK/FK` → jalankan 8 query.

Process (EN): drop+recreate DB (idempotent) → `saveAsTable` → add PK/FK → run 8 queries.

Output (real): `bigdata_etl` → `dim_client` 150,000×15, `dim_date` 12×7, `dim_macro` 12×12,
`fact_credit_monthly` 900,000×22.

Temuan analitik (8 query) / analytical findings:

| Query | Hasil ringkas / key result |

|---|---|

| Q1 overall | default rate **0.2212** |

| Q2 sex | female 0.2439 > male 0.1991 |

| Q3 education | university 0.2504 tertinggi; others 0.0752 terendah |

| Q4 marriage | married 0.2661 tertinggi |

| Q5 age | 40–49 0.2451 tertinggi; 60+ 0.1568 terendah |

| Q6 tren bulan | `credit_utilization` naik 0.306→0.478 (Apr→Sep) |

| Q7 default vs makro/bulan | FX 31.48→32.92; default rate konstan 0.2212 |

| Q8 korelasi | `corr` ≈ **0.0** (lihat catatan di bawah) |

Catatan jujur / honest note (ID): label `default_payment_next_month` bersifat **per-klien**,
didenormalisasi ke 6 baris bulanan → default rate **sama tiap bulan** (variansi nol) → korelasi
dengan makro = 0. Untuk analisis makro yang bermakna, pakai grain klien × bulan-referensi.
(EN): the default label is **client-level**, denormalised across 6 monthly rows → constant
monthly default rate → zero correlation with macro. Use a client × reference-month grain for a
meaningful macro correlation.

BAGIAN B — JALUR ELT / PART B — ELT PATH

B1. Extract+Load → tabel raw Hive / raw Hive tables

Input: `raw/synthetic_credit_clients.csv` (150k) + `raw/fred_*_2005.json`.

Kode / Code: `elt_pipeline/extract_load.py`.

Proses (ID): muat **apa adanya** (hanya nama kolom dibuat aman untuk warehouse) ke
`bigdata_elt.raw_credit` & `raw_macro` (long: `series_id`, `obs_date`, `value`). **Tanpa** unpivot/fitur.

Process (EN): load **as-is** (only column identifiers made warehouse-safe) into raw tables; no
unpivot/features. Idempotent (drop+recreate DB).

Output (real): `raw_credit` 150,000×25; `raw_macro` 48 baris (4 seri × 12 bulan).

Metadata: `elt_pipeline/logs/load_metadata.json`.

B2. Transform di dalam warehouse / in-warehouse transform

Input: `bigdata_elt.raw_credit`, `raw_macro`.

Kode / Code: `elt_pipeline/transform.sql` (dijalankan `run_transform.py`).

Proses (ID): murni SQL set-based — **kontras** dengan ETL (Python baris-per-baris):

- **Unpivot di SQL** via `LATERAL VIEW stack(6, ...)` → `elt_fact_monthly`.
- **Window functions**: `LAG`, running `AVG`, `RANK` → `elt_client_trends`.
- **OLAP GROUPING SETS** → `elt_default_by_demographic` (per dimensi + grand total).
- Pivot makro (`CASE ... MAX`) → `elt_macro_monthly`; join → `elt_default_vs_macro`.

```
LAG(bill_amt) OVER (PARTITION BY id ORDER BY date_key) AS prev_bill_amt, RANK() OVER (PARTITION BY date_key ORDER BY bill_amt DESC) AS bill_rank_in_month ... GROUPING SETS ((sex_label), (education_label), (marriage_label), (age_band), ());
```

Output (real): `elt_macro_monthly` 12 · `elt_fact_monthly` **900,000** · `elt_client_trends` **900,000** · `elt_default_by_demographic` **17** · `elt_default_vs_macro` 6.

B3. Dashboard KPI (dua jalur / both paths)

Input: `bigdata_etl.*` (ETL) dan `bigdata_elt.elt_*` (ELT).

Kode / Code: `warehouse/dashboard_kpis.sql` (via `dashboard/build_dashboard_kpis.py`).

Proses (ID): materialisasi `kpi_*` **identik** di kedua DB (`USING PARQUET`) → Tableau baca `bigdata_etl.kpi_*`, Power BI baca `bigdata_elt.kpi_*`.

Output terverifikasi beeline / beeline-verified: `kpi_overall` 0.2212/150,000; korelasi 0/0/0.

BAGIAN C — KONTRAS ETL vs ELT / PART C — ETL vs ELT CONTRAST

Aspek / Aspect	ETL	ELT
Lokasi transform	Di luar warehouse (PySpark)	Di dalam warehouse (Hive SQL)
Gaya / style	Baris-per-baris (clean, cap IQR, fitur ML)	Set-based (stack, window, GROUPING SETS)
Unpivot	<code>union 6 select</code> di Spark	<code>LATERAL VIEW stack()</code> di SQL
Output	star schema <code>bigdata_etl</code>	tabel analitik <code>bigdata_elt</code>
Beban transform	Hulu (sebelum load)	Hilir (sesudah load)
Data mentah	tersedia setelah transform	langsung query-able setelah load

Instrumentasi / instrumentation: tiap tahap memakai `common.metrics.stage_timer` → `logs/pipeline_metrics.jsonl`; `analysis/compare_pipelines.py` merangkum runtime/memori untuk laporan §9.

Ringkasan baris (bukti ≥100k) / Row counts (proof of ≥100k)

`fact_credit_monthly` (ETL) = `elt_fact_monthly` (ELT) = **900,000** baris (150,000 nasabah × 6 bulan Apr–Sep 2005) — jauh di atas minimum 100,000.